

The Ultimate Web Development Handbook



What is MongoDB?

Introduction

MongoDB is a **NoSQL document-oriented database** designed for modern application development. It stores data in flexible, JSON-like documents, which makes it easy to work with dynamic or semi-structured data.

Unlike traditional relational databases (like MySQL or PostgreSQL), MongoDB does **not use tables or rows**. Instead, it uses:

- **Databases** → which contain
- **Collections** → which contain
- **Documents** (individual records in JSON/BSON format)

Key Features of MongoDB

- **Document-Based Storage:** Data is stored in BSON (binary JSON) documents.
- **Schema-less:** Each document can have a different structure.
- **High Performance:** Built for high read/write throughput.
- **Horizontal Scalability:** Supports sharding to handle large datasets.
- **Rich Query Language:** Supports nested queries, filters, and aggregation.

MongoDB vs Relational Databases

Feature	MongoDB	Relational DB (e.g., MySQL)
Data Model	Document	Table/Row
Schema	Flexible (Schema-less)	Fixed (Predefined Schema)
Relationships	Embedded or Referenced	Foreign Keys

Feature	MongoDB	Relational DB (e.g., MySQL)
Query Language	BSON-based	SQL
Best Use Cases	Real-time apps, analytics	Financial systems, complex joins

When to Use MongoDB

MongoDB is ideal for use cases like:

- Content management systems (CMS)
- Social media applications
- Product catalogs or inventory systems
- Real-time analytics dashboards
- IoT and sensor data storage

Conclusion

MongoDB is a powerful, flexible, and scalable database solution suited for applications where data structure may evolve over time or performance at scale is critical.

Setting Up MongoDB

Local Installation (Optional for Beginners)

To install MongoDB on your system:

On Windows

1. Go to [MongoDB Community Download Center](#).
2. Download the MSI installer for your version.
3. Follow the installation steps and **enable MongoDB as a service**.
4. Use the terminal to run:

```
mongod
```

This starts the MongoDB server.

On macOS (Using Homebrew)

```
brew tap mongodb/brew  
brew install mongodb-community  
brew services start mongodb-community
```

On Linux (Ubuntu Example)

```
sudo apt update  
sudo apt install -y mongodb  
sudo systemctl start mongodb
```

To check if MongoDB is running:

```
mongo
```

This will open the MongoDB shell if it's installed correctly.

Using MongoDB Atlas (Cloud Setup)

MongoDB Atlas is the easiest way to get started without installing anything.

Steps to Create a Cluster:

1. Visit <https://www.mongodb.com/cloud/atlas>
2. Sign up and create a **free cluster** (Shared Tier).
3. Choose cloud provider and region.
4. Create a **username and password** for database access.
5. Whitelist your IP address or allow all IPs (`0.0.0.0/0`) for testing.
6. Get your connection string, which looks like:

```
mongodb+srv://<username>:<password>@cluster0.mongodb.net/myDatabase?
retryWrites=true&w=majority
```

MongoDB Compass (Optional GUI)

MongoDB Compass is a GUI client to visually interact with your databases.

- Download from: <https://www.mongodb.com/products/compass>
- Connect using the same Atlas URI or local URI like:

```
mongodb://localhost:27017
```

You can:

- View collections and documents
- Run queries
- Inspect schema visually

Connecting to MongoDB with Node.js

Install the official MongoDB driver:

```
npm install mongodb
```

Sample connection code:

```
const { MongoClient } = require('mongodb');

const uri = 'your_connection_string';
const client = new MongoClient(uri);

async function run() {
  await client.connect();
  const db = client.db('test');
  const collection = db.collection('students');
  const result = await collection.find().toArray();
  console.log(result);
  await client.close();
}

run();
```

Create and Read Documents

In MongoDB, data is stored in **documents** (which are JSON-like objects) inside **collections**. You can perform **Create** and **Read** operations using simple methods.

This section will cover:

- `insertOne()` , `insertMany()`
- `find()` , `findOne()`
- Basic filters and projections

Note: All code examples in this section are written for **MongoDB Compass** (MongoDB Shell syntax). You can run these directly in the MongoDB Compass shell or mongosh.

Setting Up Sample Data

Before we start, let's create a school database with students and teachers. Run this in MongoDB Compass:

```
// Switch to school database
use school

// Insert sample teachers
db.teachers.insertMany([
  {
    _id: ObjectId("507f1f77bcf86cd799439011"),
    name: 'Dr. Kumar',
    subject: 'MongoDB',
    experience: 5
  },
  {
    _id: ObjectId("507f1f77bcf86cd799439012"),
```

```

    name: 'Prof. Sharma',
    subject: 'Node.js',
    experience: 8
  },
  {
    _id: ObjectId("507f1f77bcf86cd799439013"),
    name: 'Ms. Patel',
    subject: 'Express',
    experience: 3
  }
])

// Insert sample students with teacher references
db.students.insertMany([
  {
    name: 'Ali',
    age: 22,
    course: 'MongoDB',
    enrolled: true,
    teacherId: ObjectId("507f1f77bcf86cd799439011"),
    grades: [85, 90, 88]
  },
  {
    name: 'Sara',
    age: 20,
    course: 'Node.js',
    enrolled: true,
    teacherId: ObjectId("507f1f77bcf86cd799439012"),
    grades: [92, 88, 95]
  },
  {
    name: 'Ahmed',
    age: 24,
    course: 'Express',
    enrolled: true,
    teacherId: ObjectId("507f1f77bcf86cd799439013"),
    grades: [78, 82, 85]
  },
  {
    name: 'Fatima',

```

```
    age: 21,
    course: 'MongoDB',
    enrolled: true,
    teacherId: ObjectId("507f1f77bcf86cd799439011"),
    grades: [95, 93, 97]
  },
  {
    name: 'Ravi',
    age: 23,
    course: 'Node.js',
    enrolled: false,
    teacherId: ObjectId("507f1f77bcf86cd799439012"),
    grades: [70, 75, 72]
  }
])
```

Inserting Documents

insertOne

Use this to insert a single document into a collection.

```
db.students.insertOne({
  name: 'Priya',
  age: 19,
  course: 'MongoDB',
  enrolled: true,
  teacherId: ObjectId("507f1f77bcf86cd799439011"),
  grades: [88, 91, 89]
})
```

insertMany

Use this to insert multiple documents at once.

```
db.students.insertMany([
  {
    name: 'Kabin',
    age: 20,
    course: 'Node.js',
    enrolled: true,
    teacherId: ObjectId("507f1f77bcf86cd799439012"),
    grades: [84, 87, 86]
  },
  {
    name: 'Zara',
    age: 22,
    course: 'Express',
    enrolled: true,
    teacherId: ObjectId("507f1f77bcf86cd799439013"),
    grades: [90, 92, 94]
  }
])
```

Reading Documents

findOne

Returns the first document that matches the filter.

```
db.students.findOne({ name: 'Ali' })
```

find

Returns all matching documents. In MongoDB Compass, results are automatically displayed.

```
db.students.find({ course: 'MongoDB' })
```

To get all students:

```
db.students.find({})
```

Projections

Use projections to include or exclude specific fields.

```
// Include only name and age (exclude _id)
db.students.find({}, { _id: 0, name: 1, age: 1 })
```

```
// Include name, course, and grades
db.students.find({}, { name: 1, course: 1, grades: 1 })
```

Other Options

Limiting Results

```
db.students.find().limit(3)
```

Sorting Results

```
// Sort by age (descending)
db.students.find().sort({ age: -1 })
```

Combining Operations

```
// Find MongoDB students, show only name and grades, sorted by age
db.students.find(
  { course: 'MongoDB' },
```

```
{ name: 1, grades: 1, _id: 0 }  
).sort({ age: 1 })
```

Working with Teachers Collection

```
// Find all teachers  
db.teachers.find()  
  
// Find teacher by subject  
db.teachers.findOne({ subject: 'Node.js' })  
  
// Find experienced teachers (more than 5 years)  
db.teachers.find({ experience: { $gt: 5 } })
```

Summary

- Use `insertOne()` and `insertMany()` to add data
- Use `findOne()` and `find()` to read data
- Use projections to control which fields are returned
- Use `.sort()` and `.limit()` to customize results
- All code runs directly in MongoDB Compass shell

Update and Delete Documents

MongoDB provides powerful methods to **update** or **remove** documents from a collection.

This section covers:

- `updateOne()` , `updateMany()` , `$set` , `$inc`
- `deleteOne()` , `deleteMany()`
- `replaceOne()`
- Understanding `ObjectId`

Note: All code examples are for MongoDB Compass shell (mongosh).

Updating Documents

updateOne

Updates the **first** document that matches the filter.

```
// Change Ali's course to Advanced MongoDB
db.students.updateOne(
  { name: 'Ali' },
  { $set: { course: 'Advanced MongoDB' } }
)
```

updateMany

Updates **all** documents that match the filter.

```
// Add 1 year to age of all enrolled students
db.students.updateMany(
```

```

    { enrolled: true },
    { $inc: { age: 1 } }
  )

// Update all MongoDB students to have a new teacher
db.students.updateMany(
  { course: 'MongoDB' },
  { $set: { teacherId: ObjectId("507f1f77bcf86cd799439011") } }
)

```

Common Update Operators

- `$set` : Sets the value of a field
- `$unset` : Removes a field
- `$inc` : Increments a numeric field
- `$push` : Adds an item to an array
- `$pull` : Removes an item from an array
- `$addToSet` : Adds unique items to an array

Examples:

```

// Add a new grade to Sara's grades array
db.students.updateOne(
  { name: 'Sara' },
  { $push: { grades: 96 } }
)

// Remove enrolled field from Ravi
db.students.updateOne(
  { name: 'Ravi' },
  { $unset: { enrolled: "" } }
)

// Increment teacher's experience by 1
db.teachers.updateOne(
  { name: 'Dr. Kumar' },

```

```
{ $inc: { experience: 1 } }  
)
```

Replacing a Document

replaceOne

Replaces the entire document with a new one (except the `_id`).

```
db.students.replaceOne(  
  { name: 'Ravi' },  
  {  
    name: 'Ravi Kumar',  
    age: 24,  
    course: 'Python',  
    enrolled: true,  
    teacherId: ObjectId("507f1f77bcf86cd799439012"),  
    grades: [80, 85, 82],  
    email: 'ravi.kumar@school.edu'  
  }  
)
```

Deleting Documents

deleteOne

Deletes the **first** matching document.

```
// Delete a specific student  
db.students.deleteOne({ name: 'Kabir' })
```

deleteMany

Deletes all matching documents.

```
// Delete all students who are not enrolled
db.students.deleteMany({ enrolled: false })

// Delete all students with low average grades
db.students.deleteMany({
  $expr: {
    $lt: [{ $avg: "$grades" }, 75]
  }
})
```

Working with ObjectId

Each document in MongoDB has a unique `_id` field of type `ObjectId`.

In MongoDB Compass, ObjectId is available globally:

```
// Find a student by _id
db.students.findOne({
  _id: ObjectId("64bd2e183dd4e6402f10388f")
})

// Update a teacher by _id
db.teachers.updateOne(
  { _id: ObjectId("507f1f77bcf86cd799439011") },
  { $set: { office: "Room 301" } }
)
```

Practical Examples

Update Student's Teacher

```
// Move all Express students to a different teacher
db.students.updateMany(
  { course: 'Express' },
  { $set: {
    teacherId: ObjectId("507f1f77bcf86cd799439012"),
    course: 'Advanced Express'
  }}
)
```

Bulk Grade Update

```
// Add bonus points to all students of a specific teacher
db.students.updateMany(
  { teacherId: ObjectId("507f1f77bcf86cd799439011") },
  { $push: { grades: 5 } } // Add 5 bonus points
)
```

Summary

- Use `updateOne()` or `updateMany()` with operators like `$set`, `$inc`, `$push`
- Use `replaceOne()` to completely replace documents
- Use `deleteOne()` or `deleteMany()` to remove documents
- `ObjectId` is globally available in MongoDB Compass
- Always verify your filters before running update/delete operations

Query Operators and Filtering

MongoDB provides powerful operators to filter and search documents in flexible ways.

In this section, you will learn how to use:

- Comparison operators (`$gt` , `$lt` , `$eq` , `$ne` , `$in` , `$nin`)
- Logical operators (`$or` , `$and` , `$not` , `$nor`)
- Array and embedded field queries
- Sorting and pagination

Note: All code examples are for MongoDB Compass shell (mongosh).

Comparison Operators

`$gt`, `$gte`, `$lt`, `$lte`

Find students older than 21:

```
db.students.find({ age: { $gt: 21 } })
```

Find students aged between 18 and 25:

```
db.students.find({ age: { $gte: 18, $lte: 25 } })
```

Find teachers with more than 5 years experience:

```
db.teachers.find({ experience: { $gt: 5 } })
```

\$eq, \$ne

Find students who are **not enrolled**:

```
db.students.find({ enrolled: { $ne: true } })
```

\$in, \$nin

Find students enrolled in either “MongoDB” or “Node.js”:

```
db.students.find({ course: { $in: ['MongoDB', 'Node.js'] } })
```

Find teachers NOT teaching Express or Python:

```
db.teachers.find({ subject: { $nin: ['Express', 'Python'] } })
```

Logical Operators

\$or

Find students enrolled in “Python” **or** age less than 20:

```
db.students.find({
  $or: [
    { course: 'Python' },
    { age: { $lt: 20 } }
  ]
})
```

\$and (default behavior)

Find students aged above 20 **and** enrolled:

```
db.students.find({
  age: { $gt: 20 },
  enrolled: true
})
```

You can also write it explicitly:

```
db.students.find({
  $and: [
    { age: { $gt: 20 } },
    { enrolled: true }
  ]
})
```

\$not

Find students **not** enrolled in "Node.js":

```
db.students.find({
  course: { $not: { $eq: 'Node.js' } }
})
```

Complex Query Example

Find enrolled students who are either: - Taking MongoDB with age > 21, OR - Taking Node.js with high grades (average > 90)

```
db.students.find({
  enrolled: true,
  $or: [
    { course: 'MongoDB', age: { $gt: 21 } },
    {
      course: 'Node.js',
      $expr: { $gt: [{ $avg: "$grades" }, 90] }
    }
  ]
})
```

```
]
})
```

Array Queries

Working with the `grades` array in our student documents:

Matching Array Elements

Find students who scored exactly 95 in any test:

```
db.students.find({ grades: 95 })
```

\$all - Multiple Values

Find students who scored both 90 and 95 at some point:

```
db.students.find({
  grades: { $all: [90, 95] }
})
```

\$size - Array Length

Find students with exactly 3 grades recorded:

```
db.students.find({
  grades: { $size: 3 }
})
```

\$elemMatch - Complex Array Conditions

Find students with at least one grade above 90:

```
db.students.find({
  grades: { $elemMatch: { $gt: 90 } }
})
```

Working with Average Grades

Find students with average grade above 85:

```
db.students.find({
  $expr: {
    $gt: [{ $avg: "$grades" }, 85]
  }
})
```

Sorting and Pagination

sort()

Sort students by age (descending):

```
db.students.find().sort({ age: -1 })
```

Sort by multiple fields:

```
// Sort by course (ascending), then by age (descending)
db.students.find().sort({ course: 1, age: -1 })
```

skip() and limit()

Get the first 3 students:

```
db.students.find().limit(3)
```

Implement pagination (skip first 2, then get next 3):

```
db.students.find().skip(2).limit(3)
```

Combined Example

Find top 3 performing MongoDB students:

```
db.students.find({  
  course: 'MongoDB'  
}).sort({  
  grades: -1  
}).limit(3)
```

Advanced Queries with Teachers

Join-like Queries

Find all students taught by Dr. Kumar:

```
// First, find Dr. Kumar's ID  
db.teachers.findOne({ name: 'Dr. Kumar' })  
  
// Then find all students with that teacherId  
db.students.find({  
  teacherId: ObjectId("507f1f77bcf86cd799439011")  
})
```

Count Operations

Count students per course:

```
db.students.countDocuments({ course: 'MongoDB' })
```

Count all enrolled students:

```
db.students.countDocuments({ enrolled: true })
```

Summary

- Use comparison operators (`$gt` , `$lt` , `$in` , etc.) for filtering
- Combine with logical operators (`$or` , `$and`) for complex queries
- Array operators (`$all` , `$size` , `$elemMatch`) for array fields
- Use `.sort()` , `.skip()` , and `.limit()` for result control
- All queries run directly in MongoDB Compass shell